Architecture Decision Log: Local FAISS vs Qdrant Free Tier

ADR-001 | Status: Accepted | Scope: vector search for the electronics product catalog

1. Context

The catalog has about 200 electronics products stored in SQLite (*products.db*), with an FTS5 table for BM25 keyword search. Titles and descriptions are embedded with *all-MiniLM-L6-v2* (384 dimensions). We need a place to store these vectors and run semantic search. Two options were considered: a local FAISS index file, or the free tier of Qdrant Cloud.

2. Options compared

Trade-off	FAISS (on-device)	Qdrant free cloud tier
Cost	Free, no usage caps. Only local CPU/RAM is used.	Free tier is capped (small cluster, limited storage) and may require a paid plan as data grows.
Latency	In-process search, typically under 1 ms for ~200 vectors. No network hop.	Every query crosses the network; tens to hundreds of ms depending on region and connection.
Privacy and offline use	Data never leaves the device; works fully offline.	Vectors and payloads are stored on a third-party server; needs internet and API keys.
Filtering and metadata	None built in. Must filter via SQLite (price, brand) and join by id.	Native payload filtering, so vector search and filters (e.g. price < 100) run in one call.
Scaling and sharing	Limited to one machine's RAM; index file must be copied to share it.	Easy multi-user access, replication and growth to millions of vectors.
Operations	We save/load the .faiss file ourselves and keep it in sync with SQLite.	Managed service: backups, upserts and updates handled by the provider; adds vendor dependency.

3. Decision

Use FAISS on-device (*IndexIDMap* over *IndexFlatIP*, saved as *products.faiss*). At 200 vectors of 384 floats the index is about 300 KB, so an exact brute-force search is instant and needs no tuning. Vectors are normalized, so inner product equals cosine similarity, and FAISS ids are the SQLite product ids, which makes joining results back to product rows trivial.

4. Consequences

Positive: zero cost, no network dependency, offline-capable, simple reproducible rebuild (*python rebuild_all.py*). **Negative:** no built-in metadata filtering (we post-filter using SQLite), and the index must be rebuilt or updated whenever products change. **Revisit when:** the catalog grows beyond roughly 100k items, multiple users or services must query the same index, or hybrid vector-plus-filter queries become a core requirement. At that point migrating to Qdrant is straightforward because the embeddings and ids are already stored in a portable form.

Note: free-tier limits and latency figures are approximate and should be verified against Qdrant's current pricing page before any migration.